

(الف)

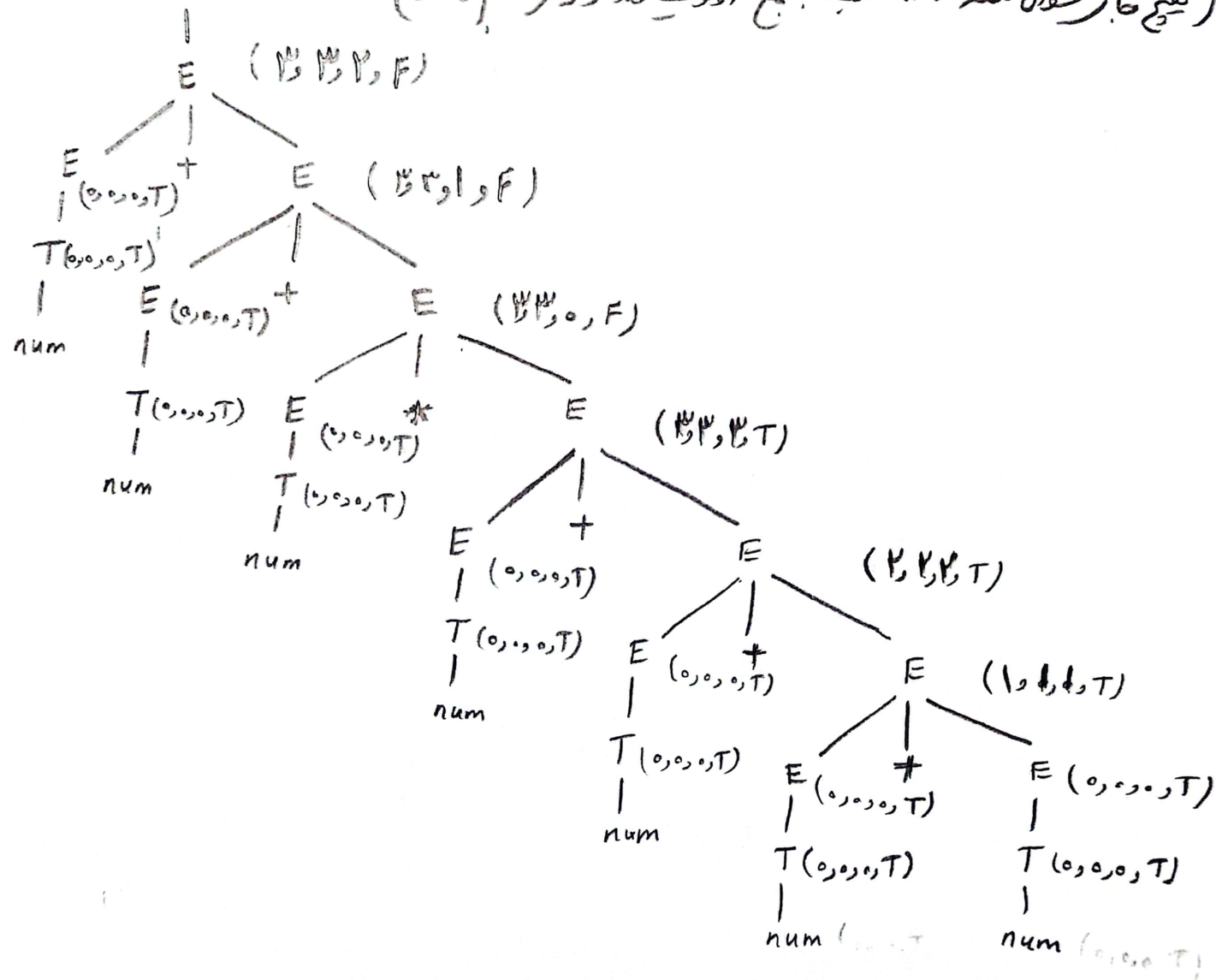
 $P \rightarrow E$ $\{ P.\text{right} = E.\text{right}, P.\text{left} = E.\text{left},$ $P.\text{isallpos} = E.\text{isallpos}, P.\text{man} = \max(E.\text{man}, E.\text{left}, E.\text{right}) \}$ $E \rightarrow E_1 + E_2$ $\{ E.\text{man} = \max(E_1.\text{man}, E_2.\text{man}, E_1.\text{right} + E_2.\text{left} + 1),$ $E.\text{isallpos} = E_1.\text{isallpos} \&\& E_2.\text{isallpos},$ $\text{if } (E_1.\text{isallpos} == \text{true}) : \{ E.\text{left} = \overset{+1}{E_1.\text{left}} + E_2.\text{left} \}$ $\text{else} : \{ E.\text{left} = E_1.\text{left} \}$ $\text{if } (E_2.\text{isallpos} == \text{true}) : \{ E.\text{right} = \overset{+1}{E_2.\text{right}} + E_1.\text{right} \}$ $\text{else} : \{ E.\text{right} = E_2.\text{right} \}$

تو این حالت ها هست
مان، left، right
E₁ یا E₂ با هم برابر است.
چون که اگر مثبت باشند.

 $E \rightarrow E_1 \times E_2$ $\{ E.\text{man} = \max(E_1.\text{man}, E_2.\text{man})$ $E.\text{isallpos} = \text{false}$ $E.\text{right} = E_2.\text{right}, E.\text{left} = E_1.\text{left} \}$ $E \rightarrow T$ $\{ E.\text{man} = T.\text{man}, E.\text{isallpos} = T.\text{isallpos},$ $E.\text{right} = T.\text{right}, E.\text{left} = T.\text{left} \}$ $T \rightarrow (E)$ $\{ T.\text{man} = E.\text{man}, T.\text{isallpos} = \text{false},$ $T.\text{right} = 0, T.\text{left} = 0 \}$ $T \rightarrow \text{num}$ $\{ T.\text{man} = 0, T.\text{right} = 0, T.\text{left} = 0, T.\text{isallpos} = \text{True} \}$

(ب) ترتیب متادیر به صورت $man, right, left, isallpos$ است که $isallpos$ و T است یا F

(هیچ بار سوال ننگنه که * نسبت به جمع اولویت دارد و اگر رسم است) (۱۳۳۷، ۴) P



(۲) الف

برنامه از main شروع می شود و داریم :

Inside main, $x=10$, $y=20$ و $z=30$

بعدش func1 اجرا می شود. در func1، $x=5$ int جدید تعریف می شود (چون x سی که global است تغییر نمی کند اما x جدید داریم). چون $y \leq x$ ($20 \leq 5$)، پس else اجرا می شود و مقدار y (که $a.b$ است) آپدیت می شود به $y = 30 - 5 = 25$. پس در print داریم:

Inside func1, $x=5$, $y=25$ و $z=30$

بعد func2، در داخل scope مربوط به func1، call می شود و چون dynamic scope است، مقدار x همان x جدید و برابر 5 است. در func2، $z > b$ ($30 > 5$)، پس else اجرا می شود و z که $a.b$ است، update می شود به $z = b + x = 5 + 5 = 10$. پس در print داریم:

Inside func2, $x=5$, $y=25$ و $z=10$

بعد در همان scope، func3، call می شود و چون $y > c$ ($25 > 5$)، پس if اجرا می شود و آن x جدید، update می شود به $x = c + y = 5 + 25 = 30$. پس در print داریم:

Inside func3, $x=30$, $y=25$ و $z=10$

(۲) ب (۱)

در ابتدا داریم : Inside main, $x=10$, $y=20$, $z=30$

func1 اجرا می شود، x محلی جدید با مقدار ۵ و y (که global بود) با (جایگزینی) $update$ می شود.
پس $y = z - x = 30 - 5 = 25$ و $print$ می شود.
به صورت کلی $update$ می شود.

Inside func1, $x=5$, $y=25$, $z=30$
به صورت محلی تعریف می شود.

با (جایگزینی) func2، $b=5$ اما $x=10$ (چون global است و دیگر در scope روی تابع func1 نیستیم)
و $else$ اجرا می شود و z گلوبال آنگاه می شود $z = b + x = 5 + 10 = 15$. پس $print$ می شود.

Inside func2, $x=10$, $y=25$, $z=15$

با (جایگزینی) func3، $c=b=5$ اما $x=10$ و چون $y > c$ ، $else$ اجرا می شود و x گلوبال
آنگاه می شود $x = c + y = 5 + 25 = 30$. پس در نهایت $print$ می شود.

Inside func3, $x=30$, $y=25$, $z=15$

$$P \rightarrow S$$

$$\{ P.count = S.count - 1 \}$$

(۳)

$$S \rightarrow PS_1$$

$$\{ S.count = S_1.count \}$$

$$S \rightarrow qS_1$$

$$\{ S.count = S_1.count \}$$

$$S \rightarrow p$$

$$\{ S.count = 0 \}$$

$$S \rightarrow q$$

$$\{ S.count = 0 \}$$

$$S \rightarrow nS_1$$

$$\{ S.count = S_1.count + 1 \}$$

$$S \rightarrow n$$

$$\{ S.count = 1 \}$$

روش: تعداد زیررشته‌ها را بر بن هر $2n$ (که بیشترین n دیگری نیست)، برابر با مقدار n در رشته منهای ۱ است.
 پس ابتدا مقدار n در رشته را حساب می‌کنیم و بعد برابر P ، منهای یک می‌کنیم.
 $\downarrow$
 root

(۱۴) الف) تنها جایی که خروجی داریم، عبارت $\frac{1}{1+n+y}$ است که باید تقسیم بر صفر شدن را دارد. برابر حالت static، مقدار n در $\frac{1}{1+n+y}$ از n کلورال گرفته شده برابر 3- است. پس y باید 2+ باشد تا خروجی را صفر نکند. اما اگر $y=2$ باشد، if اجرا شده و $else$ اجرا نمی شود. پس به کسر بر نمی خوریم و خطای رخ نمی دهد.

ب) چون dynamic scoping، چون if در scope تابع g صدا زده شده، n موجود در if لایه n در g که برابر 8- است گرفته شده. پس برابر صفر شدن خروجی $\frac{1}{1+n+y}$ ، y باید برابر 7+ باشد. و اگر هم 7+ توسط use امانه شود، همان $else$ اجرا شده و به خطای $division$ by zero در کسر می خوریم.

(الف) ۵

فرض کردیم خالی است اما ممکن است متغیرها از قبل آن باشند

```

class X {
  ←  $\Gamma_0 = \{ \}$ 
  int j; ←  $\Gamma_1$ 
  bool x[10]; ←  $\Gamma_1$ 
  void f( int a, float b[10]) {
    ←  $\Gamma_2 = \Gamma_1 \oplus \{ (a, int), (b, float[10]) \}$ 
    for (int i=0; i < a; i++) {
      ←  $\Gamma_3 = \Gamma_2 \oplus \{ (i, int) \}$ 
      x[i] = b[i] * a < z[b[i] + j]
    }
    ←  $\Gamma_3$ 
  }
  ←  $\Gamma_2$ 
}
  ←  $\Gamma_1$ 
string z;
  ←  $\Gamma_1$ 
};
  ←  $\Gamma_0$ 

```

$$\frac{\frac{n \in \text{dom}(\Gamma) \quad e \in \text{dom}(\Gamma)}{\Gamma \vdash n} \quad \Gamma \vdash e}{\Gamma \vdash n[e]}$$

(ب) ۵

(25)

$$\begin{array}{c} \frac{\frac{\frac{b \in \text{dom}(\Gamma) \quad i \in \text{dom}(\Gamma)}{\Gamma \vdash b \quad \Gamma \vdash i} \quad \frac{q \in \text{dom}(\Gamma) \quad z \in \text{dom}(\Gamma)}{\Gamma \vdash b[i] \quad \Gamma \vdash j}}{\frac{\frac{n \in \text{dom}(\Gamma) \quad i \in \text{dom}(\Gamma)}{\Gamma \vdash n \quad \Gamma \vdash i} \quad \frac{\Gamma \vdash b[i] \quad \Gamma \vdash a \quad \Gamma \vdash z \quad \Gamma \vdash b[i] + j}{\Gamma \vdash b[i] * a \quad \Gamma \vdash z[b[i] + j]}}{\Gamma \vdash n[i] \quad \Gamma \vdash b[i] * a < z[b[i] + j]}} \\ \hline \Gamma \vdash n[i] = b[i] * a < z[b[i] + j] \end{array}$$

$$\frac{\Gamma \vdash m : \text{map} \langle T_1, T_2 \rangle \quad \Gamma \vdash e_1 : T_1}{\Gamma \vdash m[e_1] : T_2}$$

(4) الف

(۶) ب) می‌تواند برابر همبسته کردن درخت استنتاج و کمتر کردن مراحل آن، و قرارش دیگر نیز برابر map افافه کرد اما برابر سان سازی درک مفهومی آن و حل آن، همین یک قانون هم کار می‌آید و کار راه‌انداز است.

$\vdash \text{intMap} : \text{map}(\text{int}, \text{int}) \quad \vdash j : \text{int} \quad \vdash \text{"key"} : \text{string}$

$\vdash \text{data} : \text{map}(\text{int}, \text{int}) \quad \vdash \text{intMap}[j] : \text{int} \quad \vdash \text{strMap} : \text{map}(\text{string}, \text{int})$

$\vdash \text{strMap} : \text{Map}(\text{string}, \text{int}) \quad \vdash \text{"result"} : \text{string} \quad \vdash \text{data}[\text{intMap}[j]] : \text{int} \quad \vdash \text{strMap}[\text{"key"}] : \text{int}$

$\vdash \text{strMap}[\text{"result"}] : \text{int} \quad \vdash \text{data}[\text{intMap}[j]] + \text{strMap}[\text{"key"}] : \text{int}$

$\vdash \text{strMap}[\text{"result"}] = \text{data}[\text{intMap}[j]] + \text{strMap}[\text{"key"}], : \text{void}$

(✓)

البته این بالا به معنی این است که ما
برای مشخص کردن واضح تر شدن کد است.

$$\frac{}{\vdash f: \text{Nat} \rightarrow \text{Nat}} \quad \frac{}{\vdash x: \text{Nat}}$$

$$\frac{}{\vdash x: \text{Nat}} \quad \frac{}{\vdash f: \text{Nat} \rightarrow \text{Nat}} \quad \frac{}{\vdash f(x): \text{Nat}}$$

$$\frac{}{\vdash f: \text{Nat} \rightarrow \text{Nat}}$$

$$\frac{}{\vdash x: \text{Nat}} \vdash f(f(x)): \text{Nat}$$

$$\frac{}{\vdash f: \text{Nat} \rightarrow \text{Nat}} \vdash \lambda x: \text{Nat}. f(f(x)): \text{Nat} \rightarrow \text{Nat}$$

$$\frac{}{\vdash y: \text{Nat}} \quad \frac{}{\vdash y: \text{Nat}} \quad \frac{}{\vdash y: \text{Nat}}$$

$$\frac{}{\vdash y: \text{Nat}} \quad \frac{}{\vdash \text{isZero}: \text{bool}} \quad \frac{}{\vdash \text{succ}: \text{Nat}} \quad \frac{}{\vdash \text{pred}: \text{Nat}}$$

$$\frac{}{\vdash y: \text{Nat}} \vdash \text{if isZero } y \text{ then succ } y \text{ else pred } y: \text{Nat}$$

$$\frac{}{\vdash (\lambda f: \text{Nat} \rightarrow \text{Nat}. \lambda x: \text{Nat}. f(f(x))): (\text{Nat} \rightarrow \text{Nat}) \rightarrow (\text{Nat} \rightarrow \text{Nat})} \quad \frac{}{\vdash (\lambda y: \text{Nat}. \text{if isZero } y \text{ then succ } y \text{ else pred } y): \text{Nat} \rightarrow \text{Nat}}$$

$$\vdash (\lambda f: \text{Nat} \rightarrow \text{Nat}. \lambda x: \text{Nat}. f(f(x))) (\lambda y: \text{Nat}. \text{if isZero } y \text{ then succ } y \text{ else pred } y): \text{Nat} \rightarrow \text{Nat}$$

الف) جب، منظور سوال یہ ہے کہ اگر $expr$ اصل، $well\ typed$ (ست)، $expr$ سادہ $well\ typed$ ہے یا نہیں
برابر مرد اول، برابر لکھ لے کر $static\ type\ check$ صحیح ہے، باید تایید e_1 و e_2 ہر دو کی $well\ typed$
(دوسرے کہ تایید e_1 و e_2 کی ہے، اور $expr$ اگر $well\ typed$ ہے، تو $well\ typed$ ہے اور e_1 تایید
 $well\ typed$ ہے۔ در واقع ایسا سازگار ہے، یک عبارت $well\ typed$ اگر وہ ایک عبارت $well\ typed$ ،
سادہ سازگار ہے کہ درست ہے، e_1 و e_2 ہم برابر یا تایید $if\ expr$ باید باشد۔)

(نکتہ دوم) لکھ بازنہ شرط اگر $true$ است یعنی e_1 اجرا ہو، اگر $static\ type\ check$
تمام حالت ہر شرط واحد کی کند کہ تایید کیا ہے یا نہیں، و گاهی بہ شرط ندارد و ہمہ حالت باید تایید کیا ہے یا نہیں
(مثال $c++$)

دوسرے دوم، در واقع یک $anonymous\ function$ (ست) کہ دوسرے n نامی کرد e_1 را با داشتن آن، حساب کند
و حالت ما می خواهیم n جابجائی n ، e_2 را به عنوان دوسرے تابع دهم، یعنی تمام جابجائی که n در حضور دارد را
با e_2 جایگزین می کنیم. حال اگر n تایید t دارد، برابر لکھ سادہ سازگار $type\ correct$ باشد، باید e_2 نیز
همان تایید t را داشته باشد. اگر خروجی تابع اصلی تایید t بود، خروجی عبارت سادہ شد t باید t
باشد (t تایید e_1 است) تا $well\ typed$ باشد

ب) برابر مرد اول، گاهی برابر سادہ سازگار درست است چون شرط $true$ است یعنی e_1 اجرا ہو
و اگر n جابجائی که e_1 اجرا شد و $llan$ هم e_1 اجرا ہو، تغییر رفتار برنامه در صورت اجرا، مشاهده
نمی کنیم اما در صورتی که منظور از تغییر رفتار سادہ سازگار باشد، همان سادہ سازگار که n جابجائی
معنی اگر e_1 و e_2 تایید کیا ہے یا نہیں، در زمان کامپایل اجرا می دهد اما عبارت سادہ شد e_1 این را
را نخواهد داد (اگر منظور از تغییر رفتار این است که، بله، تغییر رفتار دارد)

برابر مرد دوم، زمانی برابر سادہ سازگار درست است که اولاً e_2 ، $side\ effect$ نداشته باشد (مثلاً درون خود
تابع $print$ نداشته باشد). اگر n آمد در e_1 به کار گرفته باشد، $side\ effect$ دارد و e_2 (در صورت
وجود) از بین می رود. حتی مثال در صفحات بعدی

ادامه ۸

ب) برابر شدن اول، فرض کنید به نحوی در عبارت اصلی، یک $goto$ در e_1 بود که به e_2 می‌رفت، در این صورت با ساده سازی کلاً برنامه فرق خواهد کرد و موجب تغییر رفتار آن می‌شود.
برابر شدن دوم هم چند مثال می‌زنیم.

مثال اول: side effect در سطرهای اضافی. مثلاً داریم

$fun(x: int) \rightarrow x + x$ و $e_2 = (print("Hi"); 1)$
(در این حالت یکبار e_2 اجرا شده و خروجی آن به چاپ می‌آید و x مقدار می‌گیرد)

حال در عبارت ساده سازی شده داریم:

$((print("Hi"); 1) / x) (x + x)$

که در صورتی که x را با e_2 جایگزین کنیم خواهیم داشت:

$(print("Hi"); 1) + (print("Hi"); 1)$

که دو بار Hi پرینت می‌شود (به جای یکبار که در حالت اولیه بود) که این side effect اضافی است

مثال دوم: از دست رفتن side effect. مثلاً داشته باشیم

$fun(x: int) \rightarrow 10$ و $e_2 = (print("Hi"); 1)$

در حالت اصلی، یک Hi پرینت خواهد شد اما در حالت ساده شده، با جایگزین کردن x به چاپ می‌آید، چون

x در e_1 استفاده نشده، پس هیچ $print$ از Hi نداریم

(حالا فرض کنید به جای $print("Hi"); 1$ ، $while(true)$ داریم. در حالت اصلی برنامه بی‌پایان بوده و

همیشه اجرا می‌شود اما در حالت ساده سازی شده، چون x در e_1 نداریم، برنامه متوقف می‌شود (سیخ)

پس برابر شدن دوم برآز شان این است که موجب تغییر رفتار برنامه می‌شود.

۸ ج) می توان معقبه از بهینه ساز را و ساز را در کد را متنی از type checking انجام داد. (مشاهده نظر
که در بخش code generation هم خواندیم بیایم به جای $P \&\&Q$ بنویسیم $(e_1 \text{ then } e_2 \text{ else false})$ if و همان
باید short circuit را از بین و یا از branch استفاده کنیم) ولی در اغلب موارد بهتر و امن تر است که در عبارتی
و بهینه ساز را بعد از type checking انجام دهیم و دیگر مشکلاتی که متبادر گشتیم (مشکلی عبارت ill typed با آن عبارتی
چک نکردن تایید، به یک عبارت well typed تبدیل شود و ارور از بین می رود) وجود نخواهد داشت.
(در short circuit، چون معقبه از اوقات با چک کردن همان عبارت اول، به جواب می رسیم، عبارت دوم را بعد از چک نمی کنیم و
این مورد باعث افزایش سرعت ما می شود)